

MULTITHREADED CITY TRAFFIC

PRACTICAL ASSIGNMENT 1

Work done by:

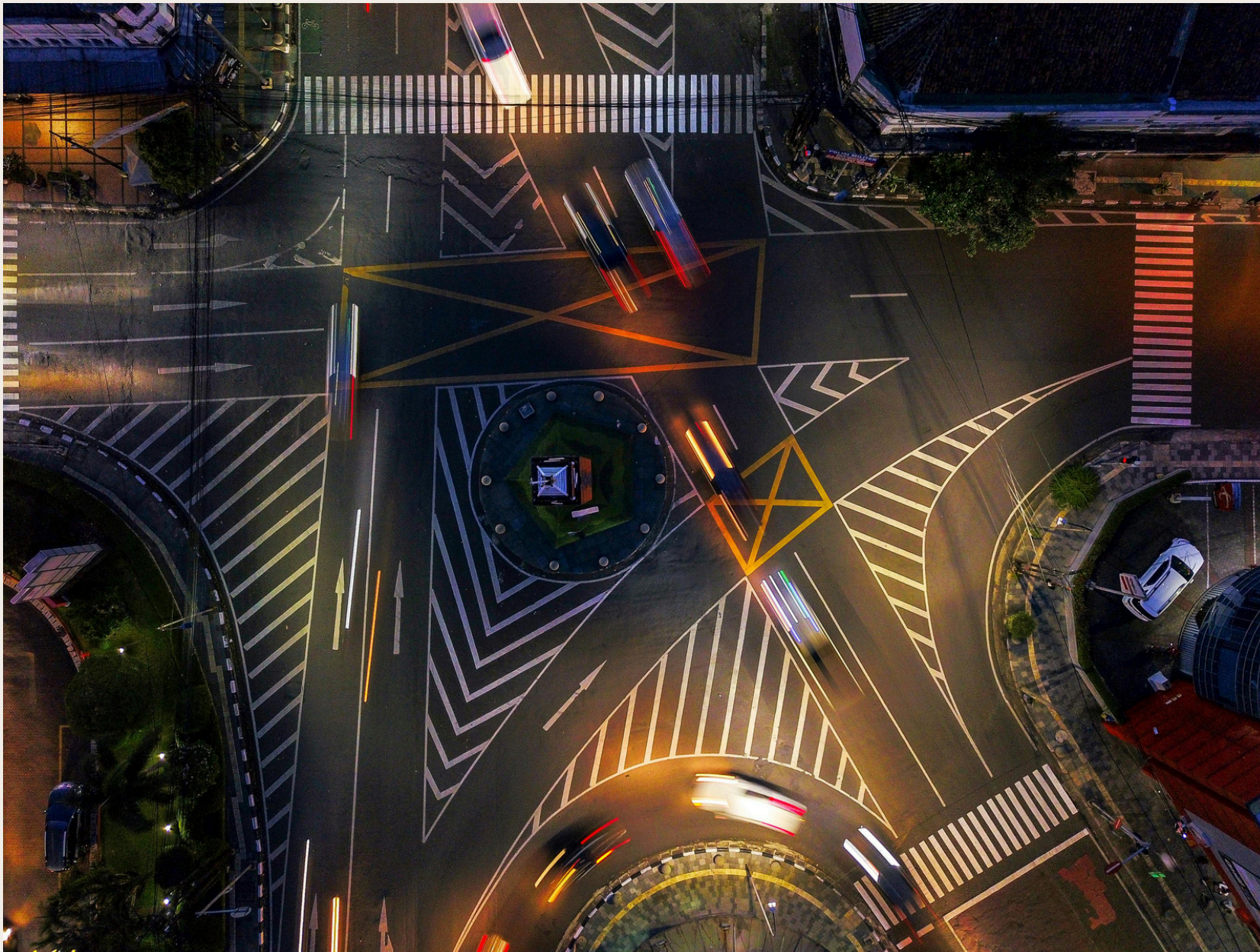
Eduardo Alves, nº104179

Pedro Ferreira, nº98620

Miguel Marques, nº103162

Distributed Systems

2025/2026

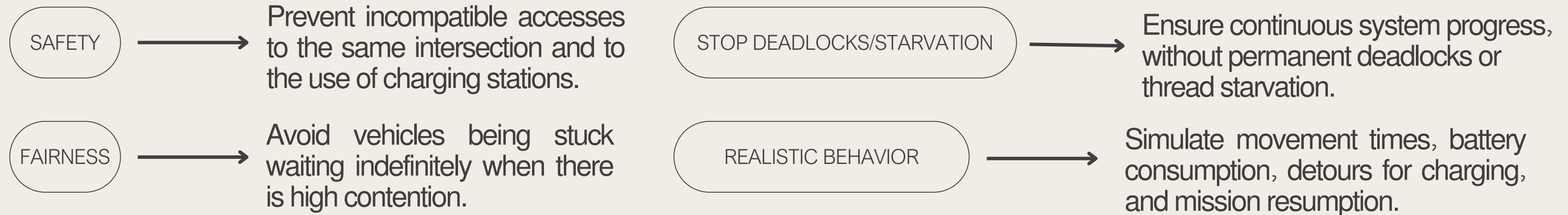


PROBLEMS AND GOALS

Why does concurrency matter in this project?

- Each vehicle is an independent thread and attempts to use shared resources at the same time.
- Intersections and chargers are contention points: without coordination, there are logical collisions, inconsistencies, and deadlocks.
- The core problem is managing concurrent access to limited resources under traffic and energy constraints.

Main Goals:



VEHICLE LIFECYCLE

1. Start

2. Pause check

3. Battery check

- If low battery and no charging station defined, find and build path to nearest charging station

4. Active path check

- If vehicle reach destination or is starting, build path to goal

5. Pick next move

- Pop direction from queue
- Calculate move to next intersection

6. Traverse intersection

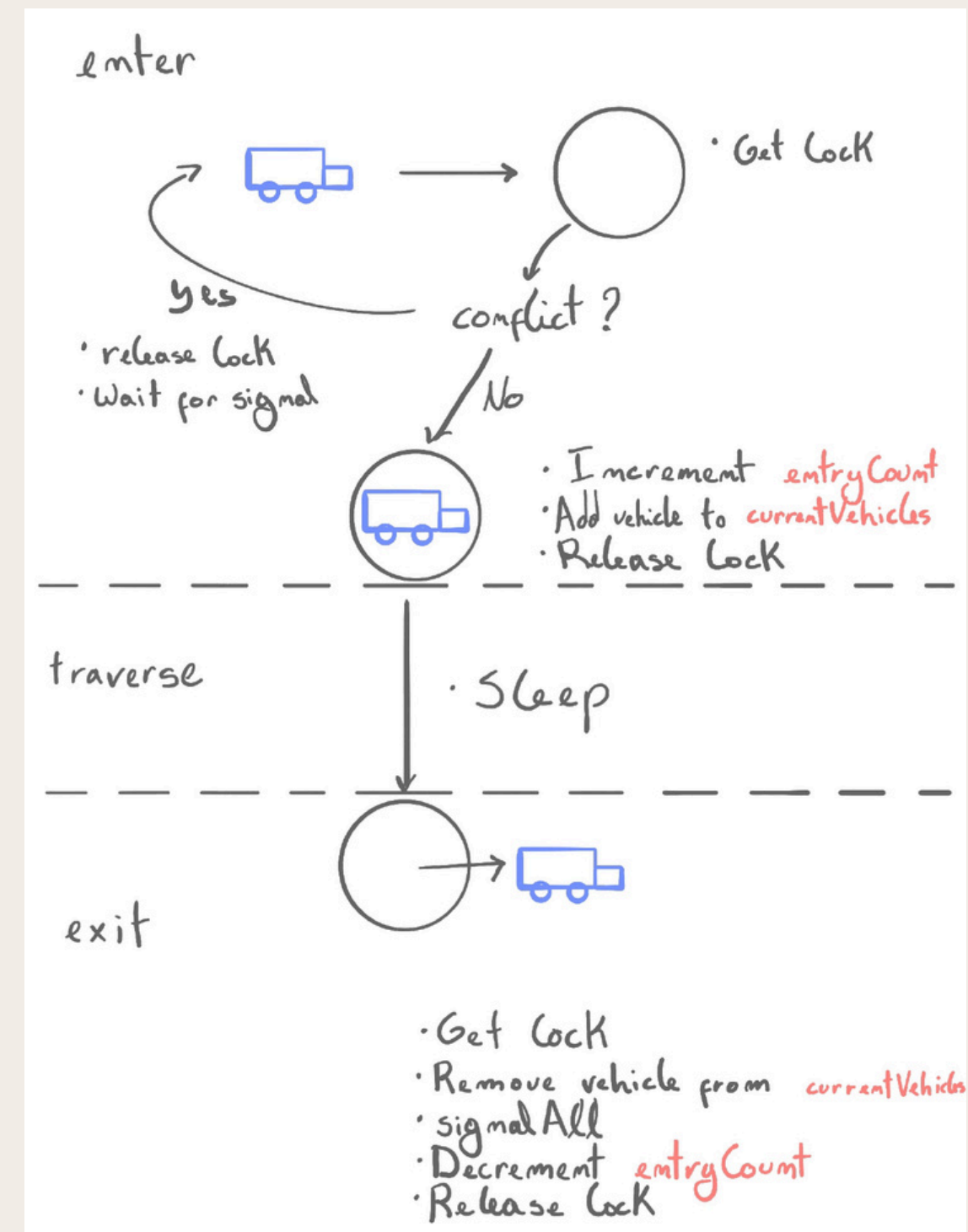
7. Charge Vehicle

8. Travel to next intersection

- Sleep (simulate travel time)
- Decrement 2-5% battery

INTERSECTION MANAGEMENT

1. **Request entry:** Wait if there is a conflicting vehicle already inside.
2. **Enter:** Register in the intersection and release the lock.
3. **Traverse:** Sleep for the traversal duration outside the lock.
4. **Exit:** De-register and wake all waiting vehicles.



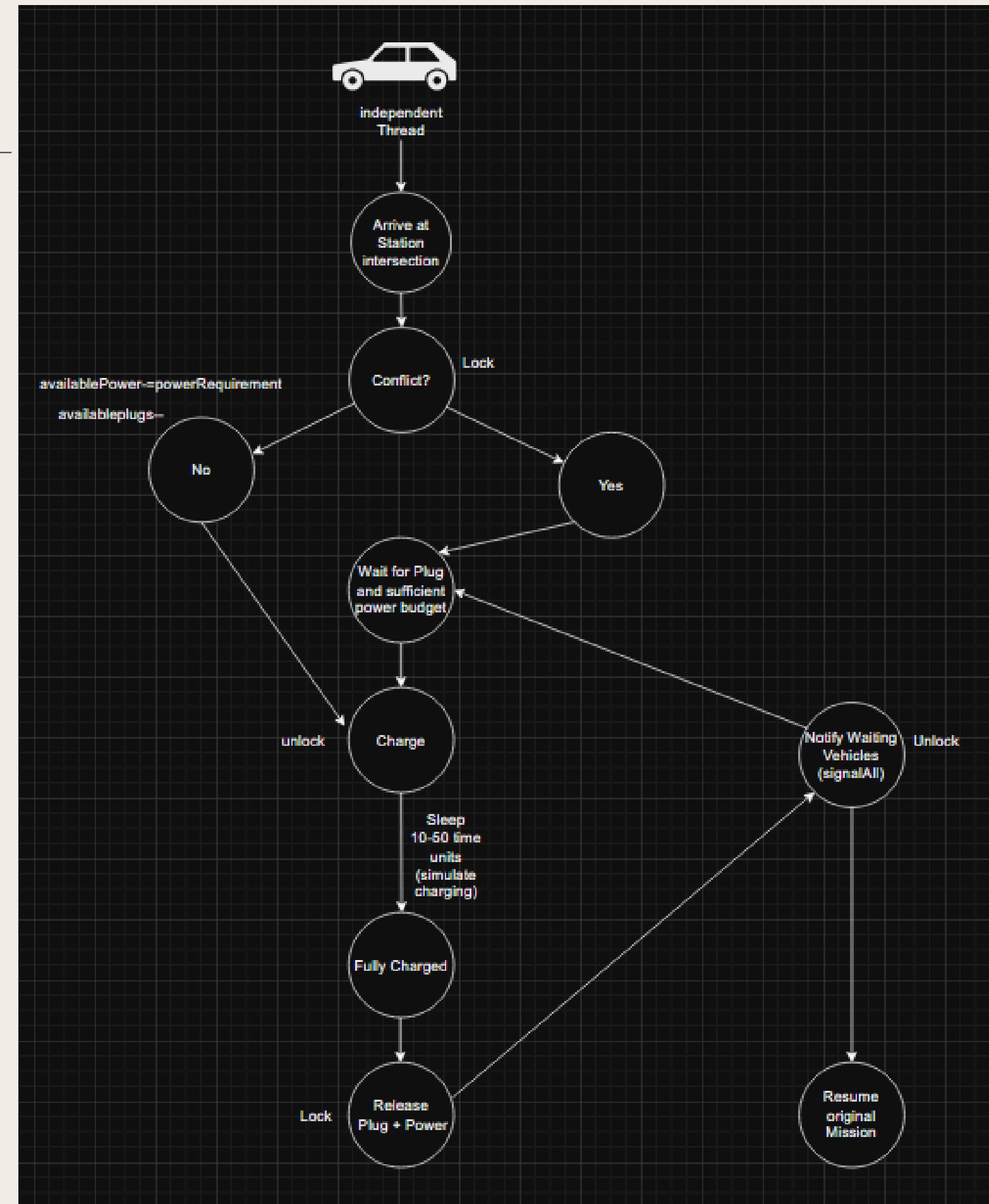
CHARGING MANAGEMENT

Core idea:

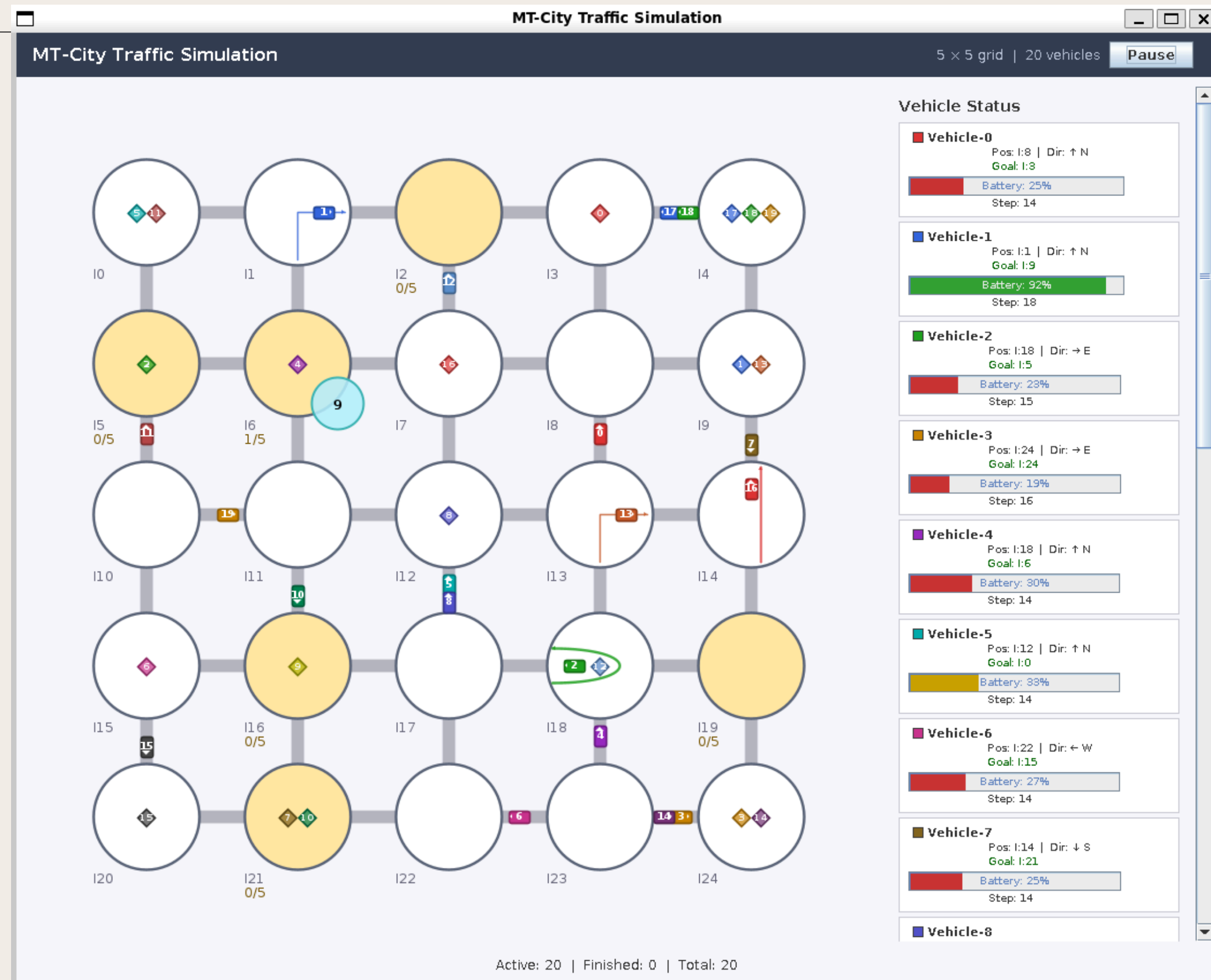
- A charging station is a shared resource with two limits: number of plugs and available power.
- A vehicle can only start charging if both conditions are true: $\text{availablePlugs} > 0$ and $\text{availablePower} \geq \text{vehicle.powerRequirement}$.
- Charging conflicts happen when many vehicles request charging concurrently and at least one of those limits is not satisfied.

How Conflicts Are Solved:

- Each thread enters, at a time, the critical section, using Lock to reserve resources.
- After reserving it, the thread unlocks so other threads can also reserve resources.
- If no resources are available, thread waits in queue for resource availability.
- After a thread finishes charging, Lock is taken again only to safely release resources and call `signalAll()`.
- `SignalAll()` wakes waiting vehicles to compete again fairly.



GRAPHICAL USER INTERFACE



DEMO